SENIOR PROJECT PB1-2026

Thai Poetry Rhyme Verification Using a Hybrid Rule-Based Approach for Education and GenAI

Project Concept

Semester 1 / 2026
TOPIC ID: PB1-2026

Submitted to

School of Information, Computer and Communication Technology
Sirindhorn International Institute of Technology
Thammasat University

Submitted date August 31, 2026

By
6622770459 Warit Yuvaniyama
6622780474 Athit Phinyachok
6622780540 Leo Colombo
6622780136 Gunn Sanguankittipun

Advisor: Dr. Prachya Boonkwan (อ.ปรัชญา บุญขวัญ)

Abstract

In Thai Klon-Paed (กลอนแปด) poetry (the eight-syllable verse form), each four-line stanza must satisfy an interlocking rhyming pattern where the rhyming syllables must share both a vowel sound (sara) and a final-consonant class (mattra). Because Thai is written without word boundaries and its orthography conceals vowels and silenced letters, automatic rhyme verification is non-trivial: neither classical rule-based checkers nor G2P-romanization checkers handle the resulting edge cases reliably. We present a hybrid rule-based verification approach: first, we improve the KhaveeVerifier algorithm: vowel analysis, final-consonant-class analysis, the rhyme test, karun silencing, and true-final detection, and ship it in PyThaiNLP 6.0 (Model B); second, we introduce Klon8Checker (Model D), which augments these rules with a gold-standard G2P override dictionary of over 4,000 entries and a fallback segmenter, recovering phonetic blind spots of the base oracle. We evaluate five checker configurations on a gold corpus of 36,474 classical stanzas (145,709 rhyme checks) and on a targeted augmentation of 11,623 instances: 10,000 adversarial negatives, 1,200 tricky positives, and 423 oracle-blind rhymes. The best configuration (D_ssg) reaches 88.5% gold recall and 86.9% F1 on the augmentation, Model B attains 100% precision on curated negatives, and D_w2p recovers 95.7% of the oracle-blind rhymes. The verifier is released as an explainable educational tool and provides a deterministic reward signal for RL-based Klon-Paed generation.

Our contributions

- **An improved rule-based verifier (Model B):** We rework the vowel analysis (check_sara), the final-consonant-class analysis (check_marttra), the rhyme test (is_sumpus), karun silencing, and true-final detection, and implement all four canonical rhyme rules. The result is merged into PyThaiNLP 6.0 as the default KhaveeVerifier. [17] https://github.com/PyThaiNLP/pythainlp/pull/1453
- **A dictionary-enhanced variant (Klon8Checker, Model D):** We augment the rules with a gold-standard G2P override dictionary of over 4,000 entries and a fallback segmenter, recovering the oracle's blind spots.
- **A large evaluation corpus:** 36,474 gold stanzas (145,709 rhyme checks) from five classical works, plus 11,623 augmentation instances: 10,000 oracle-verified negatives, 1,200 tricky positives, and 423 oracle-blind positives.
- **A systematic comparison:** Five configurations (A, B, C, D_w2p, D_ssg) are compared on precision, recall, F1, and coverage, with per-rule and per-operator error analyses.
- **Educational tool and RL-ready reward:** An explainable web tool plus a deterministic reward signal for RL-based Klon-Paed generation. The link to the hosted web tool is as follows: https://klon-pad-rhyme-checker.streamlit.app/

Acknowledgement

We thank the Vajirayana Digital Library, the PyThaiNLP maintainers for merging the KhaveeVerifier improvements, and the TLTK and KlonSuphap-LM teams.

Content

Abstract **2**
Our contributions **2**
Acknowledgement **3**
Content **4**
Project Concept **5**
Introduction **6**
Requirement Specification **7**
System Description 7
Data Preparation 7
Model Architecture 8
Evaluation 9
Precision, Recall and F1 9
Augmenting Data for Precisions 9
Speed 9
Design Specification **11**
System Architecture **11**
Evaluation Steps 13
Future Work **13**
References **14**

Project Concept

This paper has been submitted for publication at the 21st International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP 2026).

Large Language Models (LLMs) have evolved rapidly in recent years, becoming versatile assistants across domains ranging from technical coding to creative writing. Today, even small-scale LLMs can grasp and replicate the structural nuances of various poetic forms, such as the classic ABAB rhyme scheme of a sonnet or the strict 5-7-5 syllable count of a Haiku. Their success in these areas largely stems from English poetry's relative structural flexibility. Thai poetry, however, presents a significantly different challenge. It imposes stringent compositional rules, and the Thai language itself is under-resourced and introduces unique linguistic complexities absent in English, such as tonal constraints (sara) and consonant classes (mattra). Consequently, small LLMs struggle to generate Thai verses that are structurally accurate and semantically meaningful.

Our ultimate goal is to equip small LLMs with this capability using reinforcement learning (RL). This requires highly accurate reinforcement signals for the agent to learn from.

In deterministic and hard-constrained creative tasks like structured poetry, it is optimal to use Reinforcement Learning with Verifiable Rewards (RLVR) to train the LLMs. There are two main categories of verifiers for rhyme verification: deterministic and non-deterministic (ML-based). Such examples are:

- Deterministic: Rule/dictionary based which give consistent output but depend on logic implementation and have narrower coverage.
    - PyThaiNLP's KhaveeVerifier
    - TLTK G2P romanization in KlonSuphap-LM Grader
- Non-deterministic: broader coverage but results depend on exposure to training data, and the outputs are usually not consistent across similar words.
    - PyThaiNLP's W2P and ssg segmenter

Both categories have pros and cons, although currently available rhyme verifiers are erroneous or inconsistent, which are not suitable for reliable RL signals. A good verifier would provide a better RL signal for LLM training/finetuning. Therefore, we want to improve the correctness and consistency of the verification.

This project focuses on developing a hybrid rule-based system for automatically verifying rhyme correctness in Thai Klon-Paed poetry.

The proposed methods are evaluated using 36,474 stanzas of classical Thai poetry, containing 145,709 rhyme checks. An additional 11,623 targeted test instances are constructed, including 10,000 adversarial non-rhyming cases, 1,200 difficult positive rhyme cases, and 423 cases representing pronunciations that the base rule-based oracle cannot correctly identify.

Introduction

Thai orthography does not map straightforwardly onto pronunciation, and this gap becomes especially apparent in the word choice, sentence structure and overall style used by Thai poetry. The script is written without explicit word boundaries; vowels are frequently implicit; consonants or entire clusters can be silenced by the karun diacritic; and irregular words whose pronunciations do not follow the standard phonetic rules are common. Complex phonological structures, such as consonant blending and leading consonants, further obscure syllable boundaries, vowel classification, and tone. As a result, syllabification and pronunciation are particularly difficult for computational algorithms and machine learning models.

Klon-Paed is a form of Thai poetry that depends on multiple phonetic properties. Each stanza consists of four sets of seven-to-nine-syllable lines (wak); a valid rhyme requires two syllables to share both the vowel sound (sara) and the final-consonant class (mattra); four canonical rhyme rules must hold within and across stanzas; and the final syllable of every wak must participate. A single silent letter or a short-long vowel-length error is enough to break a rhyme. These constraints are strict and deterministic, which is difficult for statistical systems to consistently verify correctness without being prone to hallucinations.

A dependable automatic verifier is needed in two main domains: education and NLP research. In education, teachers and students need deterministic, explainable, hallucination-free feedback. In NLP research, training a language model to compose Klon-Paed, e.g., via reinforcement learning (RL), requires a programmatic reward process that scores rhyme correctness.

Existing available verification systems fall short of this standard. The prior version of KhaveeVerifier in PyThaiNLP 5.0.1 (Model A) relies on dictionary subword tokenization, excludes one canonical rhyme rule, and reaches only 73.4% recall on our gold corpus. G2P romanization-based checkers, such as the *word_check* grader of KlonSuphap-LM (Model C), collapse long/short vowel distinctions and produce thousands of false positives on adversarial negatives. In this paper, we address these limitations with a hybrid rule-based approach and a rigorous, adversarial evaluation.

Requirement Specification

System Description

For the final verifier model, Klon8Checker, we propose a hybrid rule-based verifier that combines the consistency of the deterministic rule-based verification and the adaptability of the probabilistic ML-based segmentation models with a curated dictionary look-up table to enhance the segmentation and reduce hallucination.

The input text will have to be broken down to form “stanzas” (บท), each stanza composed of 4 “wak”, each wak will be broken into individual “words”, each word will be broken into “syllables”, and then each syllable will be compared against the others to check the rhyming according to a rule-based rhyme scheme.

We will use a deterministic word segmenter “newmm” engine that utilizes maximal matching on dictionary corpus to obtain words from each wak.
For syllable segmentation, our hand-curated POETRY_OVERRIDES dictionary with over 4,000 entries would serve as a look-up table for syllabification.

For flexibility and adaptability, a ML-based “ssg” (Conditional Random Field model) or “w2p - word to phoneme” (Sequence-to-Sequence RNN model) would perform a sub-word tokenization to segment the syllable from word that doesn’t included in the POETRY_OVERRIDES dictionary.

The syllable would be evaluated by our rule-based check_klon() (would be merged to KhaveeVerifier PyThaiNLP 6.0), where each pair of word/syllable, according to the rhyme scheme, must rhyme with each other by passing the is_sumpus() check. The word/syllable would have to pass a preprocess step to handle the silenced karun sounds, tonemarks, and prefix-suffix/finals. Each word will be evaluated using a set of fixed rules to obtain sara and mattra. We then normalize the phonetic sound to ensure that words with matching sounds but differing in orthographic writing are correctly evaluated as rhymes. Word/syllable would rhyme if both words have the same sara and mattra, and the stanzas would be correct according to the rhyme scheme if and only if all pairs of word/syllables are correctly rhymed.

Data Preparation

Our corpus consists of 5 famous Thai poetry, namely Phra Aphai Manee, Khobut, Khunchang Khunphaen, Proverbs for Women (สุภาษิตสอนหญิง), and Phukaotong Travelling Poem (นิราศภูเขาทอง) sourced from the Vajirayana Archive. The table below provides details of the raw data.

Raw data:

Metric	Count / Value	Remarks
Files	191	Total raw files
Stanzas	36,457	36,453 4-wak stanzas + 4 3-wak openers
Waks	145,900	Total poetic lines (waks)

The preprocessing process consists of cutting the act-opening stanzas, which contain only 3 waks and rhyme mismatches, from the dataset. The data will then be transformed into a .CSV file as below; note that each column contains waks 1 through 4 in each stanza.

```
None
w1,w2,w3,w4
อันกรุงไกรใหญ่ยาวสิบเก้าโยชน์,ภูเขาโขดเป็นกำแพงบุรีศรี,สะพรึบพร้อมไพร่ฟ้าประชาชี,ชาวบุรีหรรษาสถาวร
มีเอกองค์นงลักษณ์อัครราช,พระนางนาฏนามปทุมเกสร,สนมนางแสนสุรางคนิกร,ดังกินนรน่ารักลักขณา
มีโอรสสององค์ล้วนทรงลักษณ์,ประไพพักตร์เพียงเทพเลขา,ชื่ออภัยมณีเป็นพี่ยา,พึ่งแรกรุ่นชันษาสิบห้าปี
อันกุมารศรีสุวรรณนั้นเป็นน้อง,เนื้อดังทองนพคุณจำรูญศรี,พึ่งโสกันต์ชันษาสิบสามปี,พระชนนีรักใคร่ดังนัยนา
```

The cleaned gold corpus:

Name	Files	Stanzas	Waks	r1, r2, r3	rX	Total rhyme
SuphasaetSonYing	1	200	800	600	199	799
khobut	14	1,303	5,212	3,909	1,289	5,198
khunChangKhunPhaen	43	10,543	42,172	31,629	10,500	42,129
phraAphai	132	24,341	97,364	73,023	24,206	97,229
phukaoTong	1	87	348	261	86	347
TOTAL	**191**	**36,474**	**145,896**	**109,422**	**36,280**	**145,702**

Model Architecture

ID	System	Basis	Rule set	Notes
A	Original PyThaiNLP	PyThaiNLP **5.0.1** check_klon	r1, r2, rX	dict tokenizer, no r3
B	Our Improved PyThaiNLP	PyThaiNLP **6.0** check_klon (merged KhaveeVerifier)	r1, r2, r3, rX	ssg tokenizer, improved is_sumpus
C	Kongfha word_check	KlonSuphap-LM sumpass_eval.py + word_check.py	r1, r2, r3, rX (+extra ร้-ส่ง)	tltk G2P romanisation, vowel-mattra compare

ID	System	Basis	Rule set	Notes
D_ssg	Our Klon8Checker (final model)	PyThaiNLP **6.0** check_klon + POETRY_OVERRIDES dict with ssg segmentation	r1, r2, r3, rX (same as B)	ssg tokenizer, dictionary override-enhanced segmentation, improved is_sumpus
D_w2p	Our Klon8Checker (performed worse than ssg variant)	PyThaiNLP **6.0** check_klon + POETRY_OVERRIDES dict with w2p segmentation	r1, r2, r3, rX (same as B)	w2p tokenizer, dictionary override-enhanced segmentation, improved is_sumpus

Evaluation

Precision, Recall and F1

The matics, Precision, Recall and F1, are calculated on a per-rule basis (r1, r2, r3, and rX are separately evaluated) using two distinct test datasets. It is evaluated against a gold standard corpus of 36,474 verified stanzas compiled from 5 well-known classical Thai poems.

Checker	Precision	Recall	F1 score
A	28.4%	62.7%	39.1%
B	100%	73.9%	85.0%
C	30.4%	87.1%	45.1%
D_w2p	83.4%	92.1%	87.5%
D_ssg	86.8%	87.0%	86.9%

Augmenting Data for Precisions

Since the corpus only consists of true positive rhyme instances, negative test cases are needed to evaluate precision and help checkers catch bad rhymes. A script is built to automatically generate a dataset including 11,623 negative test cases augmented from the corpus (such as swapping short/long vowels or swapping in a word with a silent letter) alongside trick positives to expose blind spots.

Speed

A script is written to record the total wall-clock time each checker takes to process the entire corpus.
Parallel scaling has been tested on the checkers to see how well they speed up when the workload is distributed across multiple cores. After testing with 1 to 10 parallel workers to identify limits, Checker A, B, and D_ssg can run smoothly on 10 workers to finish the process in less than 8 seconds, while Checker D_w2p chokes under heavy load and caps the speed at 4 workers, and gets worse with more parallel workers, coming around at 24 seconds.

Checker A (PyThaiNLP 5.0.1) is the fastest tool, finishing in just 4.4 seconds using 10 parallel workers. However, this speed is achieved by cutting corners, like relying on a basic dictionary tokenizer and skipping evaluation of the third rhyming rule (r3), which causes the high error rate.
Checker B (PyThaiNLP 6.0) is the upgraded version of A. It finishes more slowly to properly use the more accurate segmentation tool and evaluate all of the poetic rules.
Checker C (Kongfha) is exceptionally slow by design because its phonetic translation model processes text on a single thread, taking an estimated 4-8 CPU-hours (running at ~0.37-0.5 s per 8-wak unit, from 36,474-unit corpus). To make it remotely usable, engineering workarounds are required, such as using Per-wak G2P lru-cache and a pool of 10 workers running parallely, which brought the runtime down to around 49 minutes.

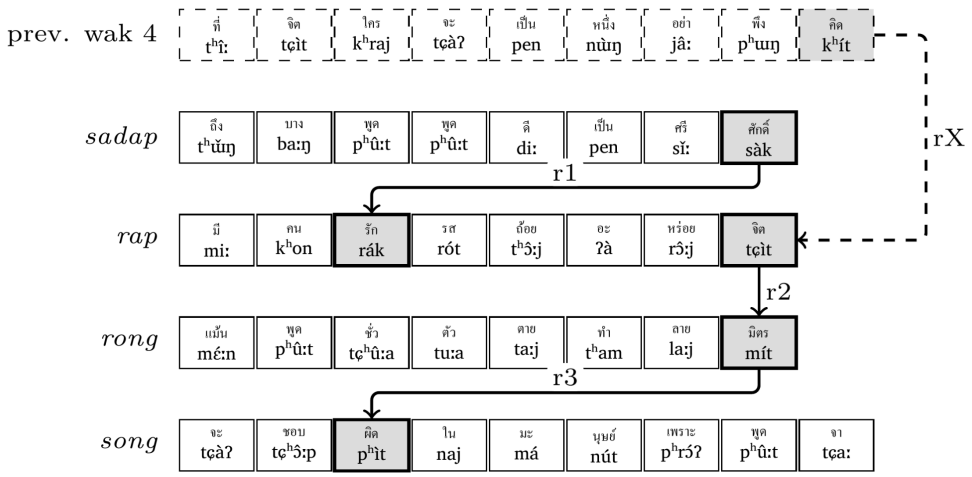
Model	**Checker**	**Number of workers**		
		1	**4**	**10**
PyThaiNLP 5.0.1	A (dict)	22.8 s	5.8 s	4.4 s
PyThaiNLP 6.0	B (ssg)	37.8 s	19.6 s	7.8 s
Klon8Checker	D_ssg	26.3s	9.5 s	6.3 s
	D_w2p	32.3 s	18.9 s	24.1 s
Kongfha word_check	C	~5 Hr too long have not run	104 min	49 min
	C (100 units sample from 36.4k units ~0.274%)	55.6 s	56.6 s	22.0 s

Design Specification

System Architecture

Four canonical rhyme rules are checked in this work, following the Klon canon as shown in the figure below:

1. r1 (sàdàp → ráp)
    - The final syllable of wak 1 must rhyme with one of the first five syllables of wak 2
2. r2 (ráp → rong)
    - The final syllable of wak 2 must rhyme with the final syllable of wak 3
3. r3 (rong → song)
    - the final syllable of wak 3 must rhyme with one of the first five syllables of wak 4
4. rX (inter-stanza)
    - The final syllable of wak 4 of the previous stanza rhymes with the final syllable of wak 2 of the current stanza.



We evaluate the performance of 5 verifier models as follows:

Model A: The original KhaveeVerifier of PyThaiNLP 5.0.1 can be accessed via this provided link: https://github.com/PyThaiNLP/pythainlp/tree/v5.0.1
Model B: Our improved rule-based verifier, which has already been merged into the KhaveeVerifier of PyThaiNLP 6.0. The model can be accessed via this provided link: https://github.com/PyThaiNLP/pythainlp/blob/main/pythainlp/khavee/core.py

Model B verifies a poem in three stages:

1. Each wak is tokenized directly into syllables with the ssg CRF segmenter.

2. For each canonical rule, the relevant syllables are extracted: the final syllable of every wak and the first five syllables of waks 2 and 4.
3. The rhyme test (is_sumpus) decides whether two syllables rhyme; a rule holds if the test succeeds for at least one permitted target.

Algorithm 1. Rhyme verification of a Klon-Paed poem (`check_klon`).

Require: poem T with waks separated by whitespace; type $k \in \{4, 8\}$
Ensure: list of rhyme errors (empty $\Leftrightarrow$ poem is valid)
$W \leftarrow$ split T into waks, grouped into stanzas of four
for each stanza (w_1, w_2, w_3, w_4) **do**
  $S_i \leftarrow \textsc{ssg}(w_i)$, $i = 1, \ldots, 4$ {syllables}
  for each rule $\rho \in \{\mathrm{r1}, \mathrm{r2}, \mathrm{r3}, \mathrm{rX}\}$ **do**
    $(a, B_\rho) \leftarrow$ source syllable and target set of rule ρ
    if no $b \in B_\rho$ satisfies $\textsc{is_sumpus}(a, b)$ **then**
      record an error for rule ρ
    end if
  end for
end for
return the list of errors

The accuracy of the Model B verifier rests on three functions:

1. check_sara (returns the canonical vowel)
2. check_marttra (returns the Royal-Institute final-consonant class), and
3. is_sumpus (normalizes arguments and checks rhyme).

Further mechanisms like handle_karun_sound_silence and _is_true_final handle silent characters and vowel/cluster ambiguity.

Model C (word_check grader of KlonSuphap-LM project, developed by Kampanat Yingseree): A G2P-based grader. Each wak (line) is romanized with the TLTK G2P model, and the romanized vowels and finals are compared. This grader was developed back in 2019 to train GPT-2-based KlonSuphap-LM. The model can be accessed via this provided link: https://github.com/Kongfha/KlonSuphap-LM

Upon investigation, we have documented two weaknesses:

1. A replace_long_short step that collapses long and short vowels together.
2. A mattra comparison that ignores the final consonant, both inflating recall at the cost of precision and TLTK(G2P model) failures that reduce coverage.

Model D (Klon8Checker): Our improvement over our general rhyme check in KhaveeVerifier of PyThaiNLP 6.0 augments these rules with a manually verified G2P override dictionary of over 4,000 entries (POETRY_OVERRIDES) to resolve irregular pronunciations that ssg or automatic rules might misidentify. We utilize two configurations:

D_ssg (fallback to ssg segmentation) and **D_w2p** (fallback to neural G2P), with **D_ssg** proving more robust.

Evaluation Steps

We report precision, recall, and F1 with Wilson 95% confidence intervals, coverage, and a conservative drop-as-fail variant for Model C. Because every augmentation instance is standalone and tests exactly one rule, the per-rule prediction is the clean metric; a stanza-level aggregate over the augmentation is only well-defined for Model B (the oracle). Model A has no r3 and is reported as N/A there. All checkers were instrumented from verbatim vendored sources (the 5.0.1 and 6.0 cores), with Model C run under Python 3.12 through a persistent pool of ten tltk workers (the full augmentation pass takes about 40 minutes; A/B/D are parallelised and finish in under a minute).

Future Work

- Evaluate and compare our verifier against the competitors.
- Using the improved Verifier to fine-tune LLMs via RLVR.
- Explore the J-space/attention matrix of our fine-tuned LLM to examine how the LLM thought and constructed the poem to the correct rhyme scheme/rules.

References

[1] PyThaiNLP Contributors, "PyThaiNLP: Thai natural language processing in Python," 2024. [Online]. Available: https://github.com/PyThaiNLP/pythainlp

[2] L. Ouyang, J. Wu, X. Jiang, et al., "Training language models to follow instructions with human feedback," in NeurIPS, 2022.

[3] P. Chormai, P. Prasertsom, and A. T. Rutherford, "AttaCut: A fast and accurate neural Thai word segmenter," arXiv preprint arXiv:1911.07056, 2019.

[4] P. Chormai, P. Prasertsom, J. Cheevaprawatdomrong, and A. T. Rutherford, "Syllable-based neural Thai word segmentation," in Proc. 28th International Conference on Computational Linguistics (COLING), 2020.

[5] T. Kittinaradorn, et al., "DeepCut: A Thai word tokenization library using deep neural network," 2019. [Online]. Available: https://github.com/rkcosmos/deepcut

[6] W. Aroonmanakun, "TLTK: Thai language toolkit," [Online]. Available: https://pypi.org/project/tltk/

[7] P. Charoenpornsawat and T. Schultz, "Example-based grapheme-to-phoneme conversion for Thai," in Proc. Interspeech, 2006.

[8] S. Saychum, S. Kongyoung, A. Rugchatjaroen, P. Chootrakool, S. Kasuriya, and C. Wutiwiwatchai, "Efficient Thai grapheme-to-phoneme conversion using CRF-based joint sequence modeling," in Proc. Interspeech, 2016.

[9] A. Rugchatjaroen, S. Saychum, S. Kongyoung, P. Chootrakool, S. Kasuriya, and C. Wutiwiwatchai, "Efficient two-stage processing for joint sequence model-based Thai grapheme-to-phoneme conversion," Speech Communication, 2019.

[10] P. Suksanguan, S. Waijanya, and N. Promrit, "The extraction of beautiful sound patterns from Sunthorn Phu's poem using machine learning technique and internal rhyme rule," 2021.

[11] K. Yingyingseree, "KlonSuphap-LM: GPT-2 for Thai Klon-Paed," Hugging Face model, 2024. [Online]. Available: https://huggingface.co/Kongfha/KlonSuphap-LM

[12] K. Yingyingseree, "PhraAphaiManee-LM," Hugging Face model, 2023. [Online]. Available: https://huggingface.co/Kongfha/PhraAphaiManee-LM

[13] T. Triyason, "Lilit to Ramakien: A five-work benchmark for LLM understanding of classical Thai literature," in Proc. ICAIT, ACM, 2026.

[14] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without fine-tuning," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[15] Vajirayana Digital Library. [Online]. Available: https://vajirayana.org

[16] A. Rutherford, "Thai literature corpora (TLC)," [Online]. Available: https://attapol.github.io/tlc.html

[17] PyThaiNLP, "Merge Pull Request #1453: Rewrite KhaveeVerifier: Fix the logic of check_sara, check_marttra, is_sumpus, handle_karun_sound_silence, and more," 2026. [Online]. Available: https://github.com/PyThaiNLP/pythainlp/pull/1453